```
!pip install -q --upgrade pip
!pip install -q "transformers>=4.41.0,<5.0.0" "datasets>=2.14.0" "accelerate" "huggingface_hub" sentencepiece

```

```
import sys, importlib
import torch, torchvision, torchaudio
import transformers, datasets
print("Python:", sys.version.split()[0])
print("Torch:", torch.__version__)
print("Torchvision:", torchvision.__version__)
print("Torchaudio:", torchaudio.__version__)
print("Transformers:", transformers.__version__)
print("Datasets:", datasets.__version__)
```

```
Python: 3.12.12
Torch: 2.8.0+cu126
Torchvision: 0.23.0+cu126
Torchaudio: 2.8.0+cu126
Transformers: 4.57.1
Datasets: 4.0.0
```

```
from transformers import pipeline
import torch
device = 0 if torch.cuda.is_available() else -1
clf = pipeline("text-classification", model="textdetox/bert-multilingual-toxicity-classifier",
               tokenizer="textdetox/bert-multilingual-toxicity-classifier",
               device=device, batch_size=1, return_all_scores=True)
print(clf("adham lo eppudina nee mokam chooskunnava"))
print(clf("raamudu manchi baaludu"))
print(clf("You are an amazing person"))
print(clf("You are an idiot"))
print(clf("pandi pirraloda"))
print(clf("what the heck"))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%  946/946 [00:00<00:00, 31.6kB/s]
model.safetensors: 100%  711M/711M [00:07<00:00, 164MB/s]
tokenizer_config.json:  1.22k/? [00:00<00:00, 57.1kB/s]
vocab.txt:  996k/? [00:00<00:00, 11.5MB/s]
tokenizer.json:  2.92M/? [00:00<00:00, 18.1MB/s]
special_tokens_map.json: 100%  125/125 [00:00<00:00, 5.89kB/s]
Device set to use cuda:0
/usr/local/lib/python3.12/dist-packages/transformers/pipelines/text_classification.py:111: UserWarning: `return_all_scores`
  warnings.warn(
[[{'label': 'LABEL_0', 'score': 0.5099256634712219}, {'label': 'LABEL_1', 'score': 0.4900743365287781}]]
[[{'label': 'LABEL_0', 'score': 0.3442077338695526}, {'label': 'LABEL_1', 'score': 0.6557922959327698}]]
[[{'label': 'LABEL_0', 'score': 0.9957939386367798}, {'label': 'LABEL_1', 'score': 0.004206082317978144}]]
[[{'label': 'LABEL_0', 'score': 0.001826774911023676 4}, {'label': 'LABEL_1', 'score': 0.9981732368469238}]]
[[{'label': 'LABEL_0', 'score': 0.28914156556129456}, {'label': 'LABEL_1', 'score': 0.7108584642410278}]]
[[{'label': 'LABEL_0', 'score': 0.0019770446233451366}, {'label': 'LABEL_1', 'score': 0.9980229139328003}]]
```

```
# ✅ Install dependencies
!pip install --quiet huggingface_hub requests tabulate

from huggingface_hub import HfApi
import requests, re
from tabulate import tabulate

# --- Model ID ---
MODEL_ID = "textdetox/bert-multilingual-toxicity-classifier"

# --- Fetch README (model card) directly from Hugging Face ---
url = f"https://huggingface.co/{MODEL_ID}/raw/main/README.md"
response = requests.get(url)
readme_text = response.text

# --- Parse performance metrics from README ---
# Look for markdown tables or inline patterns like "English | 0.9035"
```

```
pattern = re.compile(r"\|\s*([A-Za-z\s\-]+)\s*\|\s*([0-9]*\.[0-9]+)\s*\|")
matches = pattern.findall(readme_text)

metrics = []
if matches:
    for lang, score in matches:
        metrics.append((lang.strip(), float(score)))
else:
    print("⚠️ No table-style metrics found. Trying inline patterns...")
    # fallback: inline pattern like "English = 0.9035"
    inline_pattern = re.compile(r"([A-Za-z\s\-]+)\s*=\s*([0-9]*\.[0-9]+)")
    metrics = inline_pattern.findall(readme_text)

# --- Display results ---
if metrics:
    print(f"\n✅ Built-in Performance Metrics from Hugging Face model card ({MODEL_ID}):\n")
    print(tabulate(metrics, headers=["Language", "F1 Score"], tablefmt="fancy_grid"))

    scores = [float(s[1]) for s in metrics]
    print("\nSummary:")
    print(f"Average F1 Score: {sum(scores)/len(scores):.3f}")
    print(f"Best Language Performance: {max(metrics, key=lambda x: float(x[1]))[0]} ({max(scores):.3f})")
    print(f"Lowest Language Performance: {min(metrics, key=lambda x: float(x[1]))[0]} ({min(scores):.3f})")
else:
    print("❌ No metrics found in model card.")

# --- (Optional) Save README locally ---
with open("model_card.txt", "w", encoding="utf-8") as f:
    f.write(readme_text)
print("\n📄 Model card content saved as 'model_card.txt'")
```

```
✅ Built-in Performance Metrics from Hugging Face model card (textdetox/bert-multilingual-toxicity-classifier):
```

Language	F1 Score
en	0.9035
ru	0.9224
uk	0.9461
de	0.5181
es	0.7291
ar	0.5139
am	0.6316
hi	0.7268
zh	0.6703
it	0.6485
fr	0.9125
hin	0.685
he	0.8686
ja	0.8644
tt	0.617

```
Summary:
Average F1 Score: 0.744
Best Language Performance: uk (0.946)
Lowest Language Performance: ar (0.514)

📄 Model card content saved as 'model_card.txt'
```

```
# ============================================================
# 📊 AI Guardian Toxicity Evaluation - English Comments Only
# ============================================================

!pip install --quiet transformers torch scikit-learn pandas tqdm seaborn matplotlib

```

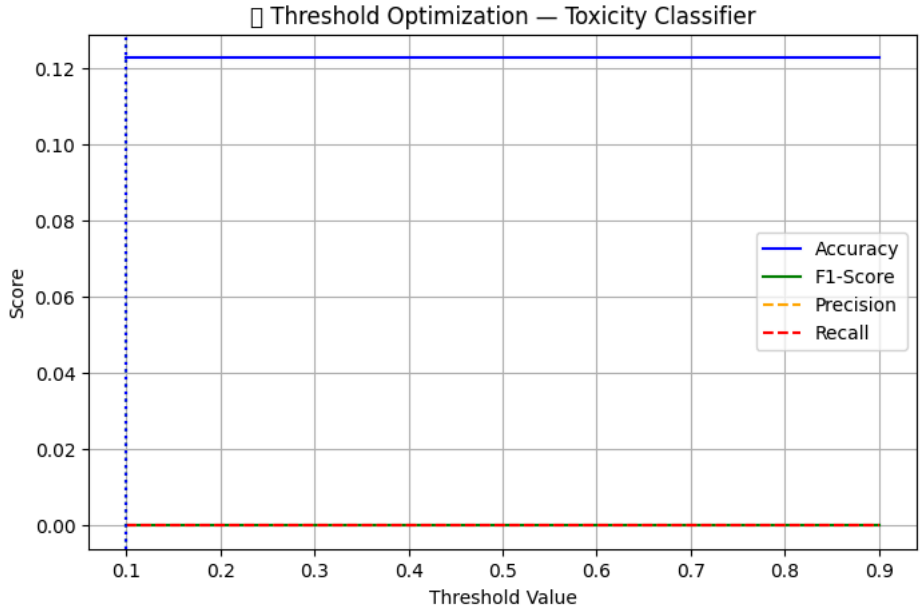
```python
import pandas as pd
import numpy as np
from tqdm import tqdm
import torch
from transformers import pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score
)
import matplotlib.pyplot as plt
import seaborn as sns
import time

# ============================================================
# 1️⃣ Load the dataset (already uploaded)
# ============================================================

file_name = "cleaned_shuffled_dataset_updated.csv"
df = pd.read_csv(file_name)

print("✅ Dataset loaded successfully!")
print("\nPreview of dataset:")
display(df.head())

cols = list(df.columns)
if len(cols) < 4:
    raise ValueError("Expected 4 columns: [Language, Toxic Word, Sentence, Label]")

lang_col, toxic_word_col, text_col, true_col = cols[:4]
print(f"\n📋 Detected Columns:\n1️⃣ {lang_col}\n2️⃣ {toxic_word_col}\n3️⃣ {text_col}\n4️⃣ {true_col}")

# Drop missing rows
df = df.dropna(subset=[text_col, true_col]).reset_index(drop=True)

# ============================================================
# 2️⃣ Load Hugging Face Toxicity Classifier
# ============================================================

device = 0 if torch.cuda.is_available() else -1
print(f"\n🔄 Loading model on {'GPU' if device == 0 else 'CPU'} ...")

clf = pipeline(
    "text-classification",
    model="textdetox/bert-multilingual-toxicity-classifier",
    tokenizer="textdetox/bert-multilingual-toxicity-classifier",
    device=device,
    return_all_scores=True,
    batch_size=16
)

# ============================================================
# 3️⃣ Run Model Inference
# ============================================================

texts = df[text_col].astype(str).tolist()
pred_labels, pred_scores = [], []

print("\n⚙️ Running toxicity predictions ...")
for txt in tqdm(texts):
    result = clf(txt)[0]
    toxic_score = 0.0
    for d in result:
        if "toxic" in d["label"].lower():
            toxic_score = d["score"]
    pred_scores.append(toxic_score)
    pred_labels.append("toxic" if toxic_score >= 0.5 else "neutral")

df["Predicted_Label"] = pred_labels
df["Toxic_Score"] = pred_scores

# Normalize true labels
def normalize_label(x):
    x = str(x).lower().strip()
    if "tox" in x:
        return "toxic"
    if "neu" in x or "non" in x:
        return "neutral"
    return x

df["True_Label_Norm"] = df[true_col].apply(normalize_label)

print("\n✅ Inference completed.")
```

```
display(df.head())

# ============================================================
# 4️⃣ Filter for English Comments Only
# ============================================================

english_df = df[df[lang_col].str.lower().str.contains("english", na=False)].copy()
if english_df.empty:
    raise ValueError("No English comments found in the dataset!")

print(f"\n🧠 Total English comments found: {len(english_df)}")

# ============================================================
# 5️⃣ Compute Performance Metrics (English Only)
# ============================================================

y_true = (english_df["True_Label_Norm"] == "toxic").astype(int).values
y_pred = (english_df["Predicted_Label"] == "toxic").astype(int).values
y_score = english_df["Toxic_Score"].values

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred, zero_division=0)
rec = recall_score(y_true, y_pred, zero_division=0)
f1 = f1_score(y_true, y_pred, zero_division=0)
cm = confusion_matrix(y_true, y_pred)

print("\n=== 📊 Model Performance (English Comments) ===")
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1-Score  : {f1:.4f}")

try:
    auc = roc_auc_score(y_true, y_score)
    print(f"ROC-AUC   : {auc:.4f}")
except:
    print("ROC-AUC   : Not available (single-class data).")

print("\n🔹 Confusion Matrix (rows=True, cols=Predicted):")
print(cm)

print("\n🔹 Detailed Classification Report:")
print(classification_report(y_true, y_pred, target_names=["Neutral", "Toxic"], zero_division=0))

# ============================================================
# 6️⃣ Visualize Confusion Matrix
# ============================================================

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Neutral", "Toxic"],
            yticklabels=["Neutral", "Toxic"])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix - English Comments")
plt.show()

# ============================================================
# 7️⃣ Save Results
# ============================================================

output_file = "english_comments_evaluation_results.csv"
english_df.to_csv(output_file, index=False)
print(f"\n✅ English-only evaluation results saved to: {output_file}")
```

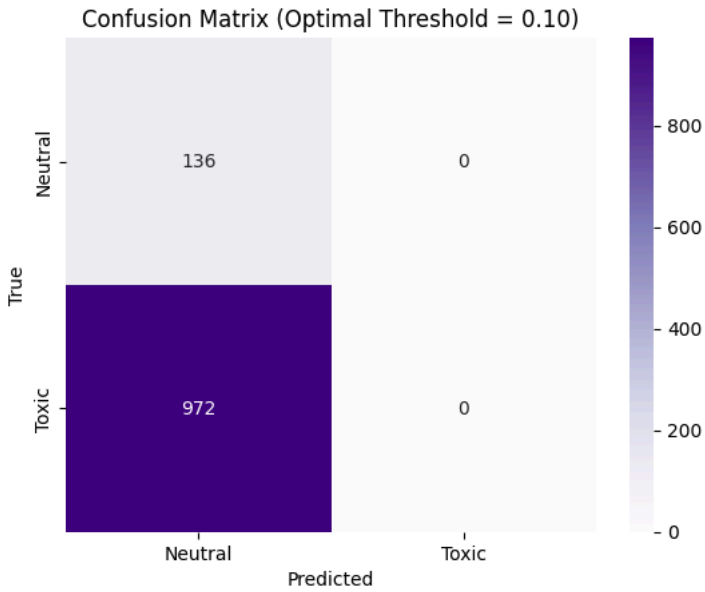
```
✅ Dataset loaded successfully!
```

```python
# ============================================================
# 🔍 AI Guardian — Automatic Threshold Tuning for Toxicity Detection
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score

# Ensure you have df already from previous code
# df contains: True_Label_Norm, Toxic_Score

# 1️⃣ Prepare clean subset with valid labels
valid_df = df[df["True_Label_Norm"].isin(["toxic", "neutral"])].copy()
y_true = (valid_df["True_Label_Norm"] == "toxic").astype(int).values
y_score = valid_df["Toxic_Score"].values

# 2️⃣ Sweep thresholds from 0.1 to 0.9
thresholds = np.arange(0.1, 0.91, 0.01)
acc_scores, f1_scores, prec_scores, rec_scores = [], [], [], []

for t in thresholds:
    y_pred = (y_score >= t).astype(int)
    acc_scores.append(accuracy_score(y_true, y_pred))
    f1_scores.append(f1_score(y_true, y_pred, zero_division=0))
    prec_scores.append(precision_score(y_true, y_pred, zero_division=0))
    rec_scores.append(recall_score(y_true, y_pred, zero_division=0))

# 3️⃣ Find best threshold (based on F1 or Accuracy)
best_f1_idx = np.argmax(f1_scores)
best_acc_idx = np.argmax(acc_scores)

best_f1_t = thresholds[best_f1_idx]
best_acc_t = thresholds[best_acc_idx]

print("\n=== 🔧 Optimal Thresholds ===")
print(f"Best Threshold (F1-score)     : {best_f1_t:.2f}  → F1 = {f1_scores[best_f1_idx]:.4f}, Acc = {acc_scores[best_f1
print(f"Best Threshold (Accuracy)     : {best_acc_t:.2f}  → Acc = {acc_scores[best_acc_idx]:.4f}, F1 = {f1_scores[best_

# 4️⃣ Visualization
plt.figure(figsize=(8,5))
plt.plot(thresholds, acc_scores, label="Accuracy", color="blue")
plt.plot(thresholds, f1_scores, label="F1-Score", color="green")
plt.plot(thresholds, prec_scores, label="Precision", color="orange", linestyle="--")
plt.plot(thresholds, rec_scores, label="Recall", color="red", linestyle="--")
plt.axvline(best_f1_t, color="green", linestyle=":")
plt.axvline(best_acc_t, color="blue", linestyle=":")
plt.title("🔍 Threshold Optimization — Toxicity Classifier")
plt.xlabel("Threshold Value")
plt.ylabel("Score")
plt.legend()
plt.grid(True)
plt.show()

# 5️⃣ Evaluate final performance at best threshold
opt_threshold = best_f1_t
print(f"\nRe-evaluating model with optimal threshold = {opt_threshold:.2f}")

y_pred_opt = (y_score >= opt_threshold).astype(int)

opt_acc = accuracy_score(y_true, y_pred_opt)
opt_prec = precision_score(y_true, y_pred_opt, zero_division=0)
opt_rec = recall_score(y_true, y_pred_opt, zero_division=0)
opt_f1 = f1_score(y_true, y_pred_opt, zero_division=0)

print("\n=== 📊 Final Optimized Performance ===")
print(f"Accuracy  : {opt_acc:.4f}")
print(f"Precision : {opt_prec:.4f}")
print(f"Recall    : {opt_rec:.4f}")
print(f"F1-Score  : {opt_f1:.4f}")

# Optional confusion matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_true, y_pred_opt)
sns.heatmap(cm, annot=True, fmt="d", cmap="Purples", xticklabels=["Neutral","Toxic"], yticklabels=["Neutral","Toxic"])
plt.title(f"Confusion Matrix (Optimal Threshold = {opt_threshold:.2f})")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

```
=== 🔧 Optimal Thresholds ===
Best Threshold (F1-score)     : 0.10  → F1 = 0.0000, Acc = 0.1227
Best Threshold (Accuracy)     : 0.10  → Acc = 0.1227, F1 = 0.0000
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128269 (\N{LEFT-POINTING MAGNIF
  fig.canvas.print_figure(bytes_io, **kw)
```




```
Re-evaluating model with optimal threshold = 0.10

=== 📊 Final Optimized Performance ===
Accuracy  : 0.1227
Precision : 0.0000
Recall    : 0.0000
F1-Score  : 0.0000
```




```
# Final stable fine-tuning (no evaluation_strategy args) — CPU/GPU safe
!pip install --quiet transformers==4.57.1 datasets evaluate scikit-learn pandas accelerate

import pandas as pd
import numpy as np
from datasets import Dataset
from sklearn.model_selection import train_test_split
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
import evaluate
import torch
```

```
import gc

print("Torch:", torch.__version__)
print("Transformers:", __import__('transformers').__version__)
print("CUDA available:", torch.cuda.is_available())

# -------------------------
# 1) Load and prepare data
# -------------------------
file_name = "cleaned_shuffled_dataset_updated.csv"
df = pd.read_csv(file_name)
cols = df.columns.tolist()
print("Detected columns:", cols)

text_col = cols[2]   # Context_or_Example_Sentence
label_col = cols[3]  # Category_of_Toxicity

# Normalize labels
df[label_col] = df[label_col].astype(str).str.lower().str.strip()
df[label_col] = df[label_col].replace({
    "non-toxic": "neutral",
    "non toxic": "neutral",
    "neutral": "neutral",
    "toxic": "toxic",
    "1": "toxic",
    "0": "neutral"
})

label2id = {"neutral": 0, "toxic": 1}
id2label = {0: "neutral", 1: "toxic"}
df = df[df[label_col].isin(label2id.keys())].reset_index(drop=True)
df["label"] = df[label_col].map(label2id)

# Split (stratified)
train_df, test_df = train_test_split(df, test_size=0.2, stratify=df["label"], random_state=42)
print(f"Train / Test sizes: {len(train_df)} / {len(test_df)}")

train_ds = Dataset.from_pandas(train_df[[text_col, "label"]])
test_ds = Dataset.from_pandas(test_df[[text_col, "label"]])

# -------------------------
# 2) Tokenize
# -------------------------
model_checkpoint = "textdetox/bert-multilingual-toxicity-classifier"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

def preprocess_function(examples):
    return tokenizer(examples[text_col], truncation=True, padding="max_length", max_length=128)

train_tokenized = train_ds.map(preprocess_function, batched=True)
test_tokenized = test_ds.map(preprocess_function, batched=True)

# Remove pandas index column if present
for ds in (train_tokenized, test_tokenized):
    if "_index" in ds.column_names:
        ds = ds.remove_columns("_index")

# -------------------------
# 3) Load model
# -------------------------
model = AutoModelForSequenceClassification.from_pretrained(
    model_checkpoint,
    num_labels=2,
    id2label=id2label,
    label2id=label2id
)

# -------------------------
# 4) Metrics
# -------------------------
accuracy = evaluate.load("accuracy")
f1 = evaluate.load("f1")
precision = evaluate.load("precision")
recall = evaluate.load("recall")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy": accuracy.compute(predictions=preds, references=labels)["accuracy"],
        "precision": precision.compute(predictions=preds, references=labels)["precision"],
        "recall": recall.compute(predictions=preds, references=labels)["recall"],
```

```
        "f1": f1.compute(predictions=preds, references=labels)["f1"]
    }

# -------------------------
# 5) Training args (compatible)
# -------------------------
training_args = TrainingArguments(
    output_dir="./aiguardian_finetuned",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=8,      # increase if you want
    learning_rate=2e-5,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100,
    save_total_limit=2,
    fp16=False,              # safe for CPU
)

# -------------------------
# 6) Trainer
# -------------------------
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=None,          # no in-train eval to avoid arg issues
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)

# -------------------------
# 7) Train
# -------------------------
print("Starting training... this can take some time on CPU.")
trainer.train()

# force garbage collection and clear cuda cache if available
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()

# -------------------------
# 8) Evaluate on test set (explicitly)
# -------------------------
print("Evaluating on test set...")
eval_results = trainer.predict(test_tokenized)
metrics = compute_metrics((eval_results.predictions, eval_results.label_ids))

print("\nFinal evaluation metrics on held-out test set:")
for k, v in metrics.items():
    print(f"{k:10s}: {v:.4f}")

# Save model/tokenizer
output_dir = "./fine_tuned_aiguardian_model"
model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)
print(f"\nSaved fine-tuned model to: {output_dir}")

# Save predictions + true labels for analysis
preds = np.argmax(eval_results.predictions, axis=-1)
out_df = test_df.copy().reset_index(drop=True)
out_df["pred_label"] = [id2label[int(p)] for p in preds]
out_df["pred_score_toxic"] = torch.nn.functional.softmax(torch.tensor(eval_results.predictions), dim=1)[:,1].numpy()
out_df.to_csv("fine_tune_test_predictions.csv", index=False)
print("Saved test predictions to fine_tune_test_predictions.csv")
```

```
Torch: 2.8.0+cu126
Transformers: 4.57.1
CUDA available: True
Detected columns: ['Language_Type', 'Toxic_Word_or_Phrase', 'Context_or_Example_Sentence', 'Category_of_Toxicity']
Train / Test sizes: 886 / 222
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json:        1.22k/?  [00:00<00:00,  133kB/s]

vocab.txt:        996k/?  [00:00<00:00,  22.8MB/s]

tokenizer.json:        2.92M/?  [00:00<00:00,  81.8MB/s]

special_tokens_map.json:  100%                                                    125/125  [00:00<00:00,  15.4kB/s]

Map:  100%                                                    886/886  [00:00<00:00,  7376.38  examples/s]

Map:  100%                                                    222/222  [00:00<00:00,  4782.68  examples/s]

config.json:  100%                                                    946/946  [00:00<00:00,  100kB/s]

model.safetensors:  100%                                                    711M/711M  [00:07<00:00,  107MB/s]

Downloading  builder  script:        4.20k/?  [00:00<00:00,  388kB/s]

Downloading  builder  script:        6.79k/?  [00:00<00:00,  565kB/s]

Downloading  builder  script:        7.56k/?  [00:00<00:00,  721kB/s]

Downloading  builder  script:        7.38k/?  [00:00<00:00,  694kB/s]
/tmp/ipython-input-618726842.py:120: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for `Tr
  trainer = Trainer(
Starting training... this can take some time on CPU.
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter: ··· ··· ··· ···
wandb: WARNING If you're specifying your api key in code, ensure this code is not shared publicly.
wandb: WARNING Consider setting the WANDB_API_KEY environment variable, or running `wandb login` from the command line.
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: ravikumarrangulk18 (ravikumarrangulk18-gokaraju-rangaraju-institute-of-engin) to https://ap
Tracking run with wandb version 0.22.2
Run data is saved locally in /content/wandb/run-20251030_140229-c4ctfvap
Syncing run haunted-scarecrow-1 to Weights & Biases (docs)
View project at https://wandb.ai/ravikumarrangulk18-gokaraju-rangaraju-institute-of-engin/huggingface
View run at https://wandb.ai/ravikumarrangulk18-gokaraju-rangaraju-institute-of-engin/huggingface/runs/c4ctfvap
[888/888 04:57, Epoch 8/8]
```

Step	Training Loss
100	0.110200
200	0.000500
300	0.010800
400	0.010400
500	0.000100
600	0.000000
700	0.000000
800	0.000200

```
Evaluating on test set...

Final evaluation metrics on held-out test set:
accuracy  : 0.9955
precision : 0.9949
recall    : 1.0000
f1        : 0.9974

Saved fine-tuned model to: ./fine_tuned_aiguardian_model
Saved test predictions to fine_tune_test_predictions.csv
```

```
# Visualization of fine-tuning results
!pip install --quiet matplotlib seaborn scikit-learn pandas

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import seaborn as sns
from sklearn.metrics import (
    confusion_matrix, classification_report, roc_curve, auc,
    precision_recall_curve, average_precision_score, accuracy_score,
    precision_score, recall_score, f1_score
)

# --- Load results (from previous fine-tuning run) ---
file = "fine_tune_test_predictions.csv"
df = pd.read_csv(file)
print("Loaded:", file)
display(df.head())

# --- Expected columns: true label, pred_label, pred_score_toxic
# Try to detect useful columns
possible_true_cols = [c for c in df.columns if c.lower().startswith(('true', 'label','category'))]
true_col = possible_true_cols[0] if possible_true_cols else None
if true_col is None:
    # fallback to common name
    true_col = 'label' if 'label' in df.columns else None

if 'pred_label' not in df.columns or 'pred_score_toxic' not in df.columns:
    raise ValueError("Expected columns 'pred_label' and 'pred_score_toxic' in CSV. Found: " + ", ".join(df.columns))

# Normalize labels to binary (toxic=1, neutral=0)
def norm(x):
    x = str(x).lower().strip()
    if 'tox' in x: return 1
    if 'neu' in x or 'non' in x: return 0
    try:
        return int(x)
    except:
        return 0

# Determine true binary labels
if true_col and true_col in df.columns:
    y_true = df[true_col].apply(norm).values
else:
    raise ValueError("True label column not found. Found candidates: " + str(possible_true_cols))

y_pred_label = df['pred_label'].astype(str).str.lower().apply(lambda x: 1 if 'tox' in x else 0).values
y_score = df['pred_score_toxic'].astype(float).values

# Basic numeric summary
n = len(df)
print(f"\nSamples: {n}")
print("True label distribution:")
unique, counts = np.unique(y_true, return_counts=True)
for u,c in zip(unique,counts):
    print(f"  {u} : {c}")

# --- Confusion Matrix
cm = confusion_matrix(y_true, y_pred_label)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Neutral', 'Toxic'], yticklabels=['Neutral','Toxic'])
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

# --- Classification Report
print("\nClassification Report:\n")
print(classification_report(y_true, y_pred_label, target_names=['Neutral', 'Toxic'], zero_division=0))

# --- ROC Curve & AUC (if both classes present)
if len(np.unique(y_true)) == 2:
    fpr, tpr, _ = roc_curve(y_true, y_score)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6,5))
    plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.3f})')
    plt.plot([0,1],[0,1],'k--', alpha=0.4)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curve')
    plt.legend(loc='lower right')
    plt.grid(True)
    plt.show()
```

```
else:
    print("\nROC curve skipped (only one class present in true labels).")

# --- Precision-Recall Curve & Average Precision
if len(np.unique(y_true)) == 2:
    prec, rec, _ = precision_recall_curve(y_true, y_score)
    ap = average_precision_score(y_true, y_score)
    plt.figure(figsize=(6,5))
    plt.plot(rec, prec, label=f'PR curve (AP = {ap:.3f})')
    plt.xlabel('Recall')
    plt.ylabel('Precision')
    plt.title('Precision-Recall Curve')
    plt.legend(loc='lower left')
    plt.grid(True)
    plt.show()
else:
    print("Precision-Recall curve skipped (only one class present).")

# --- Score distribution histogram
plt.figure(figsize=(7,4))
sns.histplot(y_score, bins=30, kde=True)
plt.title("Distribution of Toxic Scores (model output)")
plt.xlabel("Toxic score")
plt.ylabel("Count")
plt.show()

# --- Threshold sweep (accuracy, f1, precision, recall vs threshold)
thresholds = np.linspace(0.0, 1.0, 101)
accs, f1s, precs, recs = [], [], [], []

for t in thresholds:
    preds_t = (y_score >= t).astype(int)
    accs.append(accuracy_score(y_true, preds_t))
    f1s.append(f1_score(y_true, preds_t, zero_division=0))
    precs.append(precision_score(y_true, preds_t, zero_division=0))
    recs.append(recall_score(y_true, preds_t, zero_division=0))

best_f1_idx = int(np.nanargmax(f1s))
best_acc_idx = int(np.nanargmax(accs))

plt.figure(figsize=(9,5))
plt.plot(thresholds, accs, label='Accuracy')
plt.plot(thresholds, f1s, label='F1')
plt.plot(thresholds, precs, label='Precision', linestyle='--')
plt.plot(thresholds, recs, label='Recall', linestyle='--')
plt.axvline(thresholds[best_f1_idx], color='gray', linestyle=':', label=f'Best F1 t={thresholds[best_f1_idx]:.2f}')
plt.axvline(thresholds[best_acc_idx], color='brown', linestyle=':', label=f'Best Acc t={thresholds[best_acc_idx]:.2f}')
plt.xlabel("Threshold")
plt.ylabel("Score")
plt.title("Threshold Sweep")
plt.legend()
plt.grid(True)
plt.show()

print(f"\nBest F1: {f1s[best_f1_idx]:.4f} at threshold {thresholds[best_f1_idx]:.2f}")
print(f"Best Accuracy: {accs[best_acc_idx]:.4f} at threshold {thresholds[best_acc_idx]:.2f}")

# --- Per-language metrics (if language column exists)
lang_col_candidates = [c for c in df.columns if 'lang' in c.lower() or 'language' in c.lower()]
if lang_col_candidates:
    lang_col = lang_col_candidates[0]
    print(f"\nPer-language metrics using column: {lang_col}")
    per_lang = []
    for lang, group in df.groupby(lang_col):
        y_t = group[true_col].apply(norm).values
        y_s = group['pred_score_toxic'].astype(float).values
        if len(np.unique(y_t)) < 2:
            continue
        preds = (y_s >= 0.5).astype(int)
        per_lang.append({
            'language': lang,
            'count': len(group),
            'accuracy': accuracy_score(y_t, preds),
            'precision': precision_score(y_t, preds, zero_division=0),
            'recall': recall_score(y_t, preds, zero_division=0),
            'f1': f1_score(y_t, preds, zero_division=0)
        })
    if per_lang:
```